



PROCESAMIENTO DE DATOS

2º 1º Cuatrimestre



4. Clase 4

Clase N° 4: Herramientas de software para Big data

Contenido:

En la clase de hoy trabajaremos los siguientes temas:

- Apache Kafka y su uso para identificación y recolección de datos
- Arquitectura Kappa y Lambda
- Analizar casos reales de aplicación de cada arquitectura.
- Explorar tecnologías actuales asociadas a su implementación.
- Ver cómo elegir la arquitectura adecuada según diferentes necesidades de negocio.

1. Presentación

Hola y bienvenidos al curso de Procesamiento de datos.

En esta cuarta semana de clases continuaremos con herramientas de software para Big Data como es el caso de Apache Kafka que nos permite identificar y recolectar datos que luego pueden ser almacenados en un clúster Hadoop. También introduciremos arquitecturas avanzadas de Big Data. Te invito a ver el siguiente video del canal DataDemia “¿Qué es Apache Kafka?” <https://www.youtube.com/watch?v=xf12AdTYppA> para meterte en el tema y tener una introducción a la herramienta.

?

2. Desarrollo y actividades

¿Qué es Apache Kafka?

Apache Kafka (Kafka) es una plataforma de transmisión (streaming) distribuida de código abierto que permite, entre otras cosas, el desarrollo de aplicaciones basadas en eventos en tiempo real.

Apache Hadoop es un marco de software distribuido que le permite almacenar cantidades masivas de datos en un clúster de ordenadores para su uso en análisis de big data, machine learning, minería de datos y otras aplicaciones basadas en datos que procesan datos estructurados y no estructurados. Kafka se suele utilizar para crear un conducto de datos en streaming en tiempo real a un clúster de Hadoop. Estudiaremos más profundamente Apache Hadoop en la próxima clase.

Actualmente, miles de millones de fuentes de datos generan continuamente secuencias de registros de datos, incluidas secuencias de eventos. Un evento es un registro digital de una acción que ha sucedido y de la hora a la que ha sucedido. Por lo general, un evento es una acción que genera otra acción como parte de un proceso. Un cliente que realiza un pedido elige un asiento en un vuelo o envía un formulario de registro son ejemplos de eventos. En un evento no tiene por qué participar una persona; por ejemplo, que un termostato conectado informe de la temperatura en un momento dado también es un evento.

Estas secuencias ofrecen oportunidades para aplicaciones que responden a datos o eventos en tiempo real. Una plataforma de streaming permite a los desarrolladores crear aplicaciones que consumen y procesan estas secuencias de forma continua a velocidades muy elevadas, con un alto nivel de fidelidad y precisión en función del orden correcto de aparición.

LinkedIn desarrolló Kafka en 2011 como intermediario de mensajes de alto rendimiento para su propio uso, posteriormente lo pasó a código abierto y donó Kafka a la Apache Software Foundation (enlace externo a ibm.com). Actualmente, Kafka se ha convertido en la plataforma de streaming más utilizada, capaz de ingerir y procesar billones de registros al día, sin ningún retraso perceptible en el rendimiento a medida que aumentan los volúmenes. Hay varias organizaciones de la lista Fortune 500, como Target, Microsoft, AirBnB y Netflix que confían en Kafka para proporcionar a sus clientes experiencias basadas en datos en tiempo real.

¿Cómo funciona Kafka?

Kafka consta de tres prestaciones principales:

Permite que las aplicaciones publiquen secuencias de datos o eventos o se suscriban a ellas.

Almacena registros con precisión (es decir, en el orden en el que se han producido) de forma duradera y con tolerancia a errores.

Procesa registros en tiempo real (tal como se producen).

Los desarrolladores pueden aprovechar estas prestaciones de Kafka a través de cuatro API:

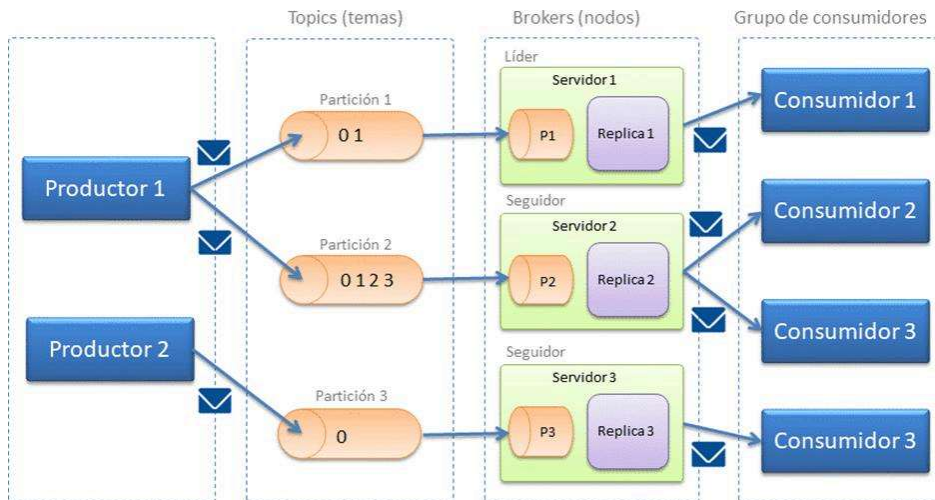
Producer API: permite que una aplicación publique una secuencia en un tema de Kafka. Un tema es un registro con nombre que almacena los registros en el orden en que se han producido, en relación entre ellos. Una vez que se escribe un registro en un tema, no se puede modificar ni eliminar, sino que permanece en el tema durante una cantidad de tiempo preconfigurada —por ejemplo, durante dos días— o hasta que se agote el espacio de almacenamiento.

Consumer API: permite que una aplicación se suscriba a uno o más temas y que ingiera y procese la secuencia almacenada en el tema. Puede trabajar con registros en el tema en tiempo real, o puede ingerir y procesar registros pasados.

Streams API: se basa en Producer API y Consumer API y agrega prestaciones complejas de procesamiento, que permiten a una aplicación realizar un procesamiento continuo de secuencias de adelante hacia atrás, específicamente, para consumir registros de uno o más temas, analizarlos, agregarlos o transformarlos según sea necesario y, finalmente, publicar las secuencias resultantes en los mismos temas u otros.

Aunque Producer API y Consumer API se pueden usar para un procesamiento de transmisión simple, es Streams API la que permite el desarrollo de aplicaciones de transmisión de datos y eventos más sofisticadas.

Connector API: permite a los desarrolladores crear conectores, que son productores o consumidores reutilizables que simplifican y automatizan la integración de una fuente de datos en un clúster de Kafka.



Arquitectura Kafka. Autor Diego Calvo

Topic (tema): Categorías en las que se clasifican los mensajes enviados a Kafka.

Producer (productor) Clientes conectados responsables de publicar los mensajes. Estos mensajes son publicados sobre uno o varios topics.

Consumer (consumidor) Clientes conectados suscritos a uno o varios topics encargados de consumir los mensajes.

Broker (nodos): Nodos que forman el cluster.

Offset: es el indicador que indica a cada consumidor el último elemento que ha leído. Esto hace que si se cae el sistema no se pierdan los datos.

¿Cómo funciona?

Apache Kafka divide cada topic (mensaje) en particiones. Cada partición es una secuencia ordenada de mensajes y cada partición es consumida por un único consumidor. Aunque se consuma la partición no se subtrae por el consumidor.

A cada topic se le puede definir un número de particiones, en función del número de servidores y de conexiones que vayamos a tener. Esto aumenta considerablemente la disponibilidad.

Cada topic tiene un offset para que cada consumidor indique que mensaje quiere que se le devuelva. A mayor número de particiones más tardará el productor (escribe mensajes) en guardar el mensaje, pero tardará menos el consumidor (lee mensajes) en recuperarlo. La idea está pensada para procesamiento en paralelo.

Cada mensaje publicado en un topic se entrega a una instancia de consumidor dentro de cada grupo de consumidores suscriptores.

Actividad 1: investigar casos de aplicación de la plataforma Kafka

Arquitecturas orientadas al dato: Data Pipelines

Las cadenas de flujos de datos o data pipelines consisten en la integración de una serie de sistemas encargados de la ejecución de un conjunto de tareas de transformación de datos descrito mediante un grafo dirigido acíclico.

Conceptualmente los data pipelines existen desde hace muchos años, ya que consisten en ejecutar unos procesos en forma de flujo con el fin de ingerir, transformar, procesar y, en su caso, persistir datos para que otro sistema pueda utilizarlos. Sin embargo, los sistemas que los implementan han ido evolucionando a lo largo del tiempo.

Tras el origen del término Data Warehouse en 1988 surge la necesidad de explotar los datos almacenados. Esto da origen a sistemas de transformación de datos en batch, que se ejecutan cada cierto tiempo generando un conjunto de datos preparados para ser consumidos. Sin embargo, estos sistemas estaban diseñados para funcionar sobre mainframes y eran poco escalables. A medida que la tecnología iba evolucionando, la cantidad de datos a almacenar y procesar aumentaba considerablemente y estos sistemas se estaban quedando obsoletos ya que debido a su diseño no podían escalar acorde a las necesidades.

No obstante, en 2004 Google ideó un modelo de programación capaz de paralelizar el procesamiento de los datos mediante cálculos parciales para luego unir los resultados, dicho modelo es el conocido MapReduce. A raíz de dicho modelo surgieron sistemas similares a los mencionados anteriormente, pero con la capacidad de escalar horizontalmente. En este aspecto el ecosistema Hadoop ha dominado el sector gracias a su sistema de ficheros basado en el Google File System, que utiliza el MapReduce para hacer procesamiento masivo de datos en batch.

Un año más tarde (2005) introdujeron los requisitos que debería tener un sistema de procesamiento de streaming en tiempo real. Este planteamiento, que no constaba más que de términos principalmente teóricos, dio lugar años más tarde a tecnologías capaces de procesar los datos a medida que iban llegando al sistema, produciendo así un resultado en tiempo real.

Este nuevo concepto ha provocado un cambio radical en las arquitecturas de los sistemas de procesamiento de datos puesto que el paradigma de programación cambia por completo al pasar de un procesamiento en batch a uno en tiempo real. Esto ha dado lugar a que las organizaciones deban plantear si llevar a cabo la reprogramación de los sistemas existentes en batch para adaptarlos a un modelo en tiempo real o la convivencia de los sistemas anteriores en batch con los nuevos en tiempo real. En 2011, Nathan Marz introdujo la arquitectura Lambda, la cual permitía la convivencia de los sistemas en batch junto con los sistemas de tiempo real. No obstante, en el año 2014, Jay Kreps cuestionaría la arquitectura Lambda, para dar paso a una nueva arquitectura que únicamente utilizará los sistemas de procesamiento en tiempo real. Esta arquitectura nacería bajo el nombre de arquitectura Kappa.

Ambas arquitecturas están compuestas por tres bloques claramente diferenciados:

- Un gestor de colas que funciona como un almacenamiento ordenado en forma de flujo de los datos de entrada. Este sistema es el que, por un lado, aporta elasticidad al sistema actuando como buffer de los datos de entrada y, por otro lado, facilita el reprocesamiento de los datos. Cada cola puede o no tener un tiempo de caducidad de los datos.

- Un motor de procesamiento de datos capaz de realizar todas las transformaciones y procesados correspondientes. Este es el núcleo de ambas arquitecturas ya que es el que contiene toda la lógica del sistema. En este caso puede haber dos tipos de motores: el motor de procesamiento en batch y el motor de procesamiento en tiempo real.
- Una base de datos que actúe como almacenamiento de los resultados obtenidos por el motor de procesamiento de datos. En este caso hay una gran cantidad de alternativas y su elección es muy dependiente de los casos de uso del sistema.

Cabe destacar que la modularidad de estos sistemas permite utilizar diferentes tecnologías por cada uno de los módulos mencionados. Así que, por ejemplo, se podría utilizar una base de datos para ciertos casos de uso y otra base de datos para otros casos de uso dentro del mismo sistema.

Actualmente ambas arquitecturas son las más recientes en el mundo del procesamiento de datos, y, por lo tanto, se va a profundizar en qué es cada una, cómo se diferencian entre sí y qué ventajas y desventajas tienen.

Un par de conceptos que tenemos que definir antes de ver las características de ambas, son el procesamiento batch y el procesamiento en streaming.

Batch hace referencia a un proceso en el que intervienen un conjunto de datos y que tiene un inicio y un fin en el tiempo.

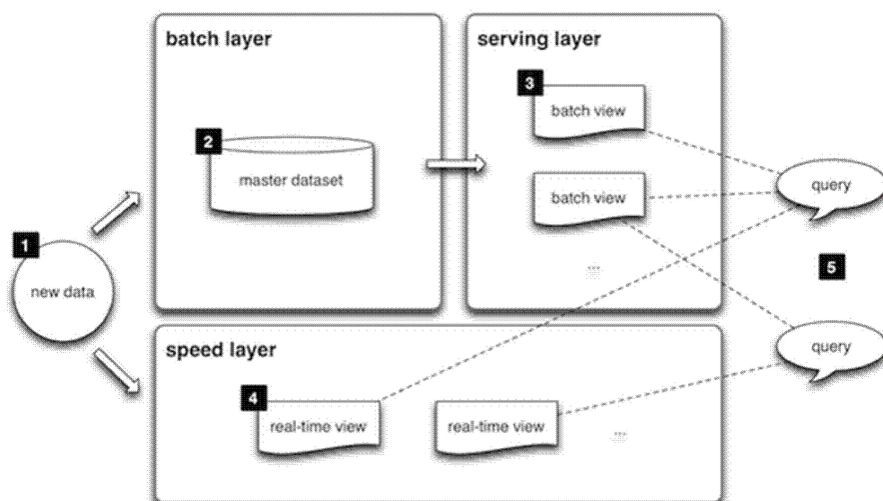
Por el contrario, decimos que un procesamiento es de tipo **streaming** cuando está continuamente recibiendo y tratando nueva información según va llegando sin tener un fin en lo referente al apartado temporal.

Arquitectura Lambda

La Arquitectura Lambda representada mediante la letra griega λ , apareció en el año 2012 y se atribuye a Nathan Marz. La definió en base a su experiencia en sistemas de tratamiento de datos distribuidos durante su etapa como empleado en las empresas Backtype y Twitter, y está inspirada en su artículo "How to beat the CAP theorem". Les comparto el link al artículo <http://nathanmarz.com/blog/how-to-beat-the-cap-theorem.html>

Su objetivo era tener un sistema robusto tolerante a fallos, tanto humanos como de hardware, que fuera linealmente escalable y que permitiese realizar escrituras y lecturas con baja latencia.

Nathan da solución a este problema creando una arquitectura cuyo diagrama de alto nivel aparece en la siguiente imagen:



Las características de la Arquitectura Lambda son:

- La nueva información recogida por el sistema se envía tanto a la capa de batch como a la capa de streaming (denominada como Speed Layer en la imagen anterior).
- En la capa batch (Batch Layer) se gestiona la información en crudo, es decir, sin modificar. Los datos nuevos se añaden a los ya existentes. Seguidamente se hace un tratamiento mediante un proceso batch cuyo resultado serán las denominadas Batch Views, que se usarán en la capa que sirve los datos para ofrecer la información ya transformada al exterior.
- La capa que sirve los datos o Serving Layer, indexa las Batch Views generadas en el paso anterior de forma que puedan ser consultadas con baja latencia.
- La capa de streaming o Speed Layer, compensa la alta latencia de las escrituras que ocurre en la serving layer y solo tiene en cuenta los datos nuevos.
- Finalmente, la respuesta a las consultas realizadas se construye combinando los resultados de las Batch Views y de las vistas en tiempo real (Real-time Views), las cuales se han generado en el paso anterior.

En resumen, este tipo de arquitectura se caracteriza por utilizar distintas capas para el procesamiento batch y el streaming.

Arquitectura Kappa

El término Arquitectura Kappa, representada por la letra κ , fue introducido en 2014 por Jay Kreps en su artículo “Questioning the Lambda Architecture”. Les comparto el link del mismo <https://www.oreilly.com/radar/questioning-the-lambda-architecture/>

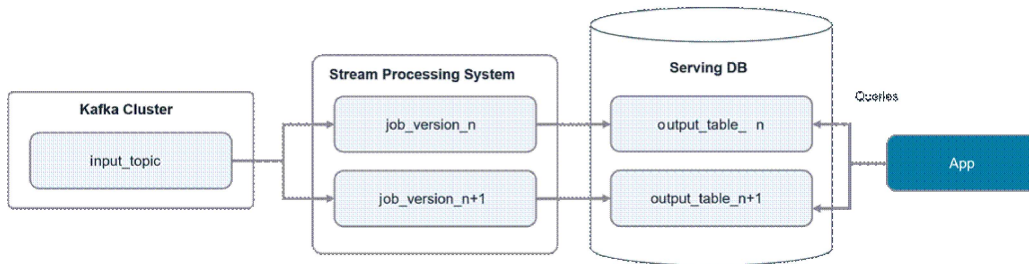
En él señala los posibles puntos “débiles” de la Arquitectura Lambda y cómo solucionarlos mediante una evolución. Su propuesta consiste en eliminar la capa batch dejando solamente la capa de streaming.

Esta capa, a diferencia de la de tipo batch, no tiene un comienzo ni un fin desde un punto de vista temporal y está continuamente procesando nuevos datos a medida que van llegando.

Como un proceso batch se puede entender como un stream acotado, podríamos decir que el procesamiento batch es un subconjunto del procesamiento en streaming.

Esta evolución consiste en una simplificación de la Arquitectura Lambda, en la que se elimina la capa batch y todo el procesamiento se realiza en una sola capa denominada de tiempo real o Real-time Layer, dando soporte a procesamientos tanto batch como en tiempo real.

El diagrama de arquitectura estaría representado por la siguiente imagen:



Podemos decir que sus cuatro pilares principales son los siguientes:

- Todo es un stream: las operaciones batch son un subconjunto de las operaciones de streaming, por lo que todo puede ser tratado como un stream.
- Los datos de partida no se modifican: los datos son almacenados sin ser transformados y las vistas se derivan de ellos. Un estado concreto puede ser recalculado puesto que la información de origen no se modifica.
- Solo existe un flujo de procesamiento: puesto que mantenemos un solo flujo, el código, el mantenimiento y la actualización del sistema se ven reducidos considerablemente.
- Posibilidad de volver a lanzar un procesamiento: se puede modificar un procesamiento concreto y su configuración para variar los resultados obtenidos partiendo de los mismos datos de entrada.
- Como requisito previo a cumplir, se tiene que garantizar que los eventos se leen y almacenan en el orden en el que se han generado. De esta forma, podremos variar un procesamiento concreto partiendo de una misma versión de los datos.

Te invito a ver el siguiente video “Patrones de arquitectura de Big Data | Arquitectura Lambda vs Kappa”

https://www.youtube.com/watch?v=waDJcSCXz_Y

Actividad 2: Utilizar alguna herramienta de I.A para obtener un resumen del video presentado.

¿Qué arquitectura se adapta mejor a nuestro problema?

Una vez visto en qué consiste cada una de las arquitecturas, viene la parte complicada que es decidir cuál es la que encaja mejor para nuestro modelo de negocio.

Como en la mayoría de los casos, se puede decir que no hay una única solución óptima para todos los problemas, lo que se suele definir mediante el término “One size does not fit all”. La Arquitectura Lambda es más versátil y es capaz de cubrir un mayor número de casos, muchos de ellos que requieren incluso procesamiento en tiempo real.

Una pregunta que debemos plantearnos para poder decidir es, ¿el análisis y el procesamiento que vamos a realizar en las capas batch y streaming es el mismo? En ese caso la opción más acertada sería la Arquitectura Kappa.

Como ejemplo real de esta arquitectura podríamos poner un sistema de geolocalización de usuarios por la cercanía a una antena de telefonía móvil.

Cada vez que se aproximase a una antena que le diese cobertura se generaría un evento. Este evento se procesaría en la capa de streaming y serviría para pintar sobre un mapa su desplazamiento respecto a su posición anterior.

Sin embargo, en otras ocasiones necesitaremos acceder a todo el conjunto de datos sin penalizar el rendimiento por lo que la Arquitectura Lambda puede ser más apropiada e incluso más fácil de implementar.

También nos inclinaremos hacia una Arquitectura Lambda si nuestros algoritmos de batch y streaming generan resultados muy distintos, como puede suceder con operaciones de procesamiento pesado o en modelos de Machine Learning.

Un caso de uso real para una arquitectura Lambda podría ser un sistema que recomiende libros en función de los gustos de los usuarios. Por un lado, tendría una capa batch encargada de para entrenar el modelo e ir mejorando las predicciones; y por otro, una capa streaming capaz de encargarse de las valoraciones en tiempo real.

Gestor de colas

A diferencia de los otros dos módulos (motor de procesamiento de datos y bases de datos), actualmente existe una única tecnología indiscutible para el gestor de colas. Dicha tecnología es Apache Kafka, un gestor de colas desarrollado por LinkedIn cuya popularidad no hace más que crecer.

Además, Apache Kafka está siendo utilizado en los pipelines de grandes compañías, como puede ser el de Yelp o el de Pinterest que utilizan Kafka como su gestor de colas.

Motor de procesamiento en Streaming

Al contrario que el gestor de colas, el procesador de streaming presenta una gran variedad de alternativas. Veamos algunos de los principales motores de procesamiento que existen actualmente:

- Spark nace del ecosistema Hadoop, como una evolución del procesamiento en batch que existía en HDFS. Spark cuenta con una gran facilidad para implementar un proceso debido al modelo MapReduce heredado de su antecesor. Es una de las soluciones más populares entre las empresas debido a su facilidad para integrarse con el ecosistema Hadoop.
- Storm nace en Twitter bajo la necesidad de computar todos los flujos de información que posee dicha red social. Storm cuenta con un gran rendimiento y sus bajas latencias, es además otra de las soluciones más elegidas por las empresas. La potencia de Storm en cuanto a latencias es prácticamente inigualable actualmente en el mercado.

Base de datos

En este apartado existe una gran cantidad de soluciones puesto que los sistemas de almacenamiento que utilizan bases de datos llevan existiendo muchos años. La elección de la tecnología en este apartado depende mucho del caso de uso, el tipo de información a almacenar y el tipo de consultas que se van a realizar al sistema.

Actividad 3: Investigar sobre motores de bases de datos y cuáles se adaptan mejor a las arquitecturas presentadas en la clase.

En resumen podemos comparar ambas arquitecturas en la siguiente tabla:

Requisito	Lambda	Kappa
Procesamiento histórico + en tiempo real	✔ Ideal	⚠ Posible pero costoso
Código y mantenimiento simples	✘ Código duplicado	✔ Flujo único

Requisito	Lambda	Kappa
Bajo volumen de datos	⚠ Overkill	✅ Más simple
Alto volumen de datos + batch pesado	✅ Robusto	⚠ Requiere tuning
Time-to-insight rápido	⚠ Puede demorar por batch	✅ Inmediato

Actividad 4:

Te invito a realizar la práctica propuesta y responder las siguientes preguntas justificando tus respuestas. En los próximos encuentros sincrónicos podremos debatir las respuestas e intercambiar ideas.

Preguntas de Opción Múltiple (Multiple Choice)**1. ¿Cuál de los siguientes casos se ajusta mejor a una arquitectura Lambda?**

- A) Monitoreo de temperatura en una planta industrial en tiempo real
- B) Seguimiento de ubicación de vehículos en una ciudad
- C) Plataforma de recomendaciones de películas con datos históricos y eventos actuales
- D) Actualización del estado de servidores con alertas inmediatas

2. ¿Qué componente de la arquitectura Lambda es responsable de entrenar modelos con grandes volúmenes de datos históricos?

- A) Serving Layer
- B) Speed Layer
- C) Batch Layer
- D) Event Queue

3. ¿Cuál de las siguientes tecnologías se asocia comúnmente con una arquitectura Kappa?

- A) Apache Hadoop
- B) Apache Spark Batch
- C) Kafka Streams
- D) RedisAI

4. En una app como Uber, que requiere reflejar la ubicación de autos en tiempo real, ¿qué arquitectura se recomienda y por qué?

- A) Lambda, porque permite combinar velocidad y precisión histórica
- B) Kappa, porque todo el flujo de datos es continuo y en tiempo real
- C) Lambda, porque permite usar Spark
- D) Ninguna, porque no aplica un modelo de arquitectura de datos

5. ¿Cuál es la ventaja principal de la arquitectura Kappa sobre la Lambda?

- A) Permite mayor variedad de tecnologías
- B) Disminuye la duplicación de lógica de procesamiento
- C) Mejora el rendimiento de consultas complejas en batch
- D) Se adapta mejor a procesamiento offline

✅ Preguntas de Verdadero o Falso

- 6. En la arquitectura Lambda, los datos se envían exclusivamente a la capa de procesamiento batch.
- 7. La arquitectura Kappa no puede manejar algoritmos que requieran análisis sobre grandes volúmenes históricos de datos.
- 8. La arquitectura Lambda es ideal para escenarios donde los modelos de Machine Learning necesitan ser reentrenados con datos históricos.
- 9. En aplicaciones como Waze, la arquitectura Lambda es preferible debido a su bajo tiempo de respuesta.
- 10. Apache Kafka puede ser usado como gestor de colas tanto en Lambda como en Kappa.

Instancia de autoevaluación: Espacio de entrega de tarea

Deberás subir un archivo PDF con las actividades propuestas resueltas. En esta instancia de autoevaluación deberás resolver las actividades propuestas en la clase y cualquier otra apreciación de los contenidos vistos que te parezca relevante.

Criterios de evaluación:

- Entregar en tiempo y forma
- Realizar un 60% de la actividad correctamente.
- Cumplir con las consignas
- Cumplir con el formato requerido.

3. Cierre

Te invito a seguir investigando sobre las tendencias actuales de arquitecturas y soluciones para implementar data pipelines. ¿Cuáles son los motores más utilizados? ¿Qué otros factores además de los que vimos en este clase pensás que pueden incidir en la decisión respecto al tipo de arquitectura a utilizar? Podés utilizar alguna herramienta de I.A para obtener ideas.

Te espero en el encuentro sincrónico.

4. Bibliografía Obligatoria:

Página oficial Kafka <https://kafka.apache.org/>

K. J., «Questioning the lambda architecture,» Julio 2014. [En línea]. Available: <https://www.oreilly.com/ideas/questioning-the-lambda-architecture>.

N. Garg (2015) Learning Apache Kafka Second Edition

S. G. Jeffrey Dean, «MapReduce: simplified data processing on large clusters,» vol. 6, 2004.

N. Marz, «How to beat the CAP theorem,» 13 Octubre 2011. [En línea]. Available: <http://nathanmarz.com/blog/how-to-beat-the-cap-theorem.html>.